

total length of the spurs? Show how to determine an optimal location in linear time.

9.3-8

The k th *quantiles* of an n -element set are the $k - 1$ order statistics that divide the sorted set into k equal-sized sets (to within 1). Give an $O(n \lg k)$ -time algorithm to list the k th quantiles of a set.

9.3-9

Describe an $O(n)$ -time algorithm that, given a set S of n distinct numbers and a positive integer $k \leq n$, determines the k numbers in S that are closest to the median of S .

9.3-10

Let $X[1 : n]$ and $Y[1 : n]$ be two arrays, each containing n numbers already in sorted order. Give an $O(\lg n)$ -time algorithm to find the median of all $2n$ elements in arrays X and Y . Assume that all $2n$ numbers are distinct.

Problems

9-1 *Largest i numbers in sorted order*

You are given a set of n numbers, and you wish to find the i largest in sorted order using a comparison-based algorithm. Describe the algorithm that implements each of the following methods with the best asymptotic worst-case running time, and analyze the running times of the algorithms in terms of n and i .

- a. Sort the numbers, and list the i largest.
- b. Build a max-priority queue from the numbers, and call EXTRACT-MAX i times.
- c. Use an order-statistic algorithm to find the i th largest number, partition around that number, and sort the i largest numbers.

9-2 *Variant of randomized selection*

Professor Mendel has proposed simplifying RANDOMIZED-SELECT by eliminating the check for whether i and k are equal. The simplified procedure is SIMPLER-RANDOMIZED-SELECT.

```

SIMPLER-RANDOMIZED-SELECT( $A, p, r, i$ )
1 if  $p == r$ 
2   return  $A[p]$  //  $1 \leq i \leq r - p + 1$  means that  $i = 1$ 
3  $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4  $k = q - p + 1$ 
5 if  $i \leq k$ 
6   return SIMPLER-RANDOMIZED-
           SELECT( $A, p, q, i$ )
7 else return SIMPLER-RANDOMIZED-
           SELECT( $A, q + 1, r, i - k$ )

```

- a. Argue that in the worst case, SIMPLER-RANDOMIZED-SELECT never terminates.
- b. Prove that the expected running time of SIMPLER-RANDOMIZED-SELECT is still $O(n)$.

9-3 Weighted median

Consider n elements $x_1, x_2, \dots, x_n$ with positive weights $w_1, w_2, \dots, w_n$ such that $\sum_{i=1}^n w_i = 1$. The **weighted (lower) median** is an element x_k satisfying

$$\sum_{x_i < x_k} w_i < \frac{1}{2}$$

and

$$\sum_{x_i > x_k} w_i \leq \frac{1}{2}.$$

For example, consider the following elements x_i and weights w_i :

|

i	1	2	3	4	5	6	7
x_i	3	8	2	5	4	1	6
w_i	0.12	0.35	0.025	0.08	0.15	0.075	0.2

For these elements, the median is $x_5 = 4$, but the weighted median is $x_7 = 6$. To see why the weighted median is x_7 , observe that the elements less than x_7 are x_1, x_3, x_4, x_5 , and x_6 , and the sum $w_1 + w_3 + w_4 + w_5 + w_6 = 0.45$, which is less than $1/2$. Furthermore, only element x_2 is greater than x_7 , and $w_2 = 0.35$, which is no greater than $1/2$.

- a. Argue that the median of $x_1, x_2, \dots, x_n$ is the weighted median of the x_i with weights $w_i = 1/n$ for $i = 1, 2, \dots, n$.
- b. Show how to compute the weighted median of n elements in $O(n \lg n)$ worst-case time using sorting.
- c. Show how to compute the weighted median in $\Theta(n)$ worst-case time using a linear-time median algorithm such as SELECT from Section 9.3.

The **post-office location problem** is defined as follows. The input is n points $p_1, p_2, \dots, p_n$ with associated weights $w_1, w_2, \dots, w_n$. A solution is a point p (not necessarily one of the input points) that minimizes the sum $\sum_{i=1}^n w_i d(p, p_i)$, where $d(a, b)$ is the distance between points a and b .

- d. Argue that the weighted median is a best solution for the one-dimensional post-office location problem, in which points are simply real numbers and the distance between points a and b is $d(a, b) = |a - b|$.
- e. Find the best solution for the two-dimensional post-office location problem, in which the points are (x, y) coordinate pairs and the distance between points $a = (x_1, y_1)$ and $b = (x_2, y_2)$ is the **Manhattan distance** given by $d(a, b) = |x_1 - x_2| + |y_1 - y_2|$.

9-4 *Small order statistics*

Let's denote by $S(n)$ the worst-case number of comparisons used by SELECT to select the i th order statistic from n numbers. Although $S(n) = \Theta(n)$, the constant hidden by the Θ -notation is rather large. When i is small relative to n , there is an algorithm that uses SELECT as a subroutine but makes fewer comparisons in the worst case.

- a.** Describe an algorithm that uses $U_i(n)$ comparisons to find the i th smallest of n elements, where

$$U_i(n) = \begin{cases} S(n) & \text{if } i \geq n/2, \\ \lfloor n/2 \rfloor + U_i(\lceil n/2 \rceil) + S(2i) & \text{otherwise.} \end{cases}$$

(Hint: Begin with $\lfloor n/2 \rfloor$ disjoint pairwise comparisons, and recurse on the set containing the smaller element from each pair.)

- b.** Show that, if $i < n/2$, then $U_i(n) = n + O(S(2i) \lg(n/i))$.
- c.** Show that if i is a constant less than $n/2$, then $U_i(n) = n + O(\lg n)$.
- d.** Show that if $i = n/k$ for $k \geq 2$, then $U_i(n) = n + O(S(2n/k) \lg k)$.

9-5 *Alternative analysis of randomized selection*

In this problem, you will use indicator random variables to analyze the procedure RANDOMIZED-SELECT in a manner akin to our analysis of RANDOMIZED-QUICKSORT in Section 7.4.2.

As in the quicksort analysis, we assume that all elements are distinct, and we rename the elements of the input array A as $z_1, z_2, \dots, z_n$, where z_i is the i th smallest element. Thus the call RANDOMIZED-SELECT($A, 1, n, i$) returns z_i .

For $1 \leq j < k \leq n$, let

$X_{ijk} = \mathbf{I} \{z_j \text{ is compared with } z_k \text{ sometime during the execution of the algorithm to find } z_i\}$.

- a.** Give an exact expression for $E[X_{ijk}]$. (*Hint:* Your expression may have different values, depending on the values of i, j , and k .)
- b.** Let X_i denote the total number of comparisons between elements of array A when finding z_i . Show that

$$E[X_i] \leq 2 \left(\sum_{j=1}^i \sum_{k=i}^n \frac{1}{k-j+1} + \sum_{k=i+1}^n \frac{k-i-1}{k-i+1} + \sum_{j=1}^{i-2} \frac{i-j-1}{i-j+1} \right).$$

- c.** Show that $E[X_i] \leq 4n$.
- d.** Conclude that, assuming all elements of array A are distinct, RANDOMIZED-SELECT runs in $O(n)$ expected time.

9-6 *Select with groups of 3*

Exercise 9.3-1 asks you to show that the SELECT algorithm still runs in linear time if the elements are divided into groups of 7. This problem asks about dividing into groups of 3.

- a.** Show that SELECT runs in linear time if you divide the elements into groups whose size is any odd constant greater than 3.
- b.** Show that SELECT runs in $O(n \lg n)$ time if you divide the elements into groups of size 3.

Because the bound in part (b) is just an upper bound, we do not know whether the groups-of-3 strategy actually runs in $O(n)$ time. But by repeating the groups-of-3 idea on the middle group of medians, we can pick a pivot that guarantees $O(n)$ time. The SELECT3 algorithm on the next page determines the i th smallest of an input array of $n > 1$ distinct elements.

- c.** Describe in English how the SELECT3 algorithm works. Include in your description one or more suitable diagrams.
- d.** Show that SELECT3 runs in $O(n)$ time in the worst case.

The worst-case linear-time median-finding algorithm was devised by Blum, Floyd, Pratt, Rivest, and Tarjan [62]. The fast randomized version is due to Hoare [218]. Floyd and Rivest [147] have developed an improved randomized version that partitions around an element recursively selected from a small sample of the elements.

```

SELECT3( $A, p, r, i$ )
1 while  $(r - p + 1) \bmod 9 \neq 0$ 
2   for  $j = p + 1$  to  $r$                                 // put the minimum into  $A[p]$ 
3     if  $A[p] > A[j]$ 
4       exchange  $A[p]$  with  $A[j]$ 
5   // If we want the minimum of  $A[p : r]$ , we're done.
6   if  $i == 1$ 
7     return  $A[p]$ 
8   // Otherwise, we want the  $(i - 1)$ st element of  $A[p + 1 : r]$ .
9    $p = p + 1$ 
10   $i = i - 1$ 
11  $g = (r - p + 1)/3$                                      // number of 3-element groups
12 for  $j = p$  to  $p + g - 1$                                // run through the groups
13   sort  $\langle A[j], A[j + g], A[j + 2g] \rangle$  in place
14 // All group medians now lie in the middle third of  $A[p : r]$ .
15  $g' = g/3$                                               // number of 3-element
                                                         subgroups
16 for  $j = p + g$  to  $p + g + g' - 1$                      // sort the subgroups
17   sort  $\langle A[j], A[j + g'], A[j + 2g'] \rangle$  in place
18 // All subgroup medians now lie in the middle ninth of  $A[p : r]$ .
19 // Find the pivot  $x$  recursively as the median of the subgroup
    medians.
20  $x = \text{SELECT3}(A, p + 4g', p + 5g' - 1, \lceil g'/2 \rceil)$ 
21  $q = \text{PARTITION-AROUND}(A, p, r, // partition around the pivot$ 
     $x)$ 
22 // The rest is just like lines 19–24 of SELECT.
23  $k = q - p + 1$ 
24 if  $i == k$ 

```

```

25  return  $A[q]$                                 // the pivot value is the answer
26 elseif  $i < k$ 
27  return SELECT3( $A, p, q - 1, i$ )
28 else return SELECT3( $A, q + 1, r, i - k$ )

```

It is still unknown exactly how many comparisons are needed to determine the median. Bent and John [48] gave a lower bound of $2n$ comparisons for median finding, and Schönhage, Paterson, and Pippenger [397] gave an upper bound of $3n$. Dor and Zwick have improved on both of these bounds. Their upper bound [123] is slightly less than $2.95n$, and their lower bound [124] is $(2 + \epsilon)n$, for a small positive constant ϵ , thereby improving slightly on related work by Dor et al. [122]. Paterson [354] describes some of these results along with other related work.

Problem 9-6 was inspired by a paper by Chen and Dumitrescu [84].

¹ As in the footnote on page 182, you can enforce the assumption that the numbers are distinct by converting each input value $A[i]$ to an ordered pair $(A[i], i)$ with $(A[i], i) < (A[j], j)$ if either $A[i] < A[j]$ or $A[i] = A[j]$ and $i < j$.

² We could also use the Akra-Bazzi method from Section 4.7, which involves calculus, to solve this recurrence. Indeed, a similar recurrence (4.24) on page 117 was used to illustrate that method.



Part III Data Structures

Introduction

Sets are as fundamental to computer science as they are to mathematics. Whereas mathematical sets are unchanging, the sets manipulated by algorithms can grow, shrink, or otherwise change over time. We call such sets *dynamic*. The next four chapters present some basic techniques for representing finite dynamic sets and manipulating them on a computer.

Algorithms may require several types of operations to be performed on sets. For example, many algorithms need only the ability to insert elements into, delete elements from, and test membership in a set. We call a dynamic set that supports these operations a *dictionary*. Other algorithms require more complicated operations. For example, min-priority queues, which Chapter 6 introduced in the context of the heap data structure, support the operations of inserting an element into and extracting the smallest element from a set. The best way to implement a dynamic set depends upon the operations that you need to support.

Elements of a dynamic set

In a typical implementation of a dynamic set, each element is represented by an object whose attributes can be examined and manipulated given a pointer to the object. Some kinds of dynamic sets assume that one of the object's attributes is an identifying *key*. If the keys are all different, we can think of the dynamic set as being a set of key values. The object may contain *satellite data*, which are carried

around in other object attributes but are otherwise unused by the set implementation. It may also have attributes that are manipulated by the set operations. These attributes may contain data or pointers to other objects in the set.

Some dynamic sets presuppose that the keys are drawn from a totally ordered set, such as the real numbers, or the set of all words under the usual alphabetic ordering. A total ordering allows us to define the minimum element of the set, for example, or to speak of the next element larger than a given element in a set.

Operations on dynamic sets

Operations on a dynamic set can be grouped into two categories: *queries*, which simply return information about the set, and *modifying operations*, which change the set. Here is a list of typical operations. Any specific application will usually require only a few of these to be implemented.

SEARCH(S, k)

A query that, given a set S and a key value k , returns a pointer x to an element in S such that $x.key = k$, or NIL if no such element belongs to S .

INSERT(S, x)

A modifying operation that adds the element pointed to by x to the set S . We usually assume that any attributes in element x needed by the set implementation have already been initialized.

DELETE(S, x)

A modifying operation that, given a pointer x to an element in the set S , removes x from S . (Note that this operation takes a pointer to an element x , not a key value.)

MINIMUM(S) and MAXIMUM(S)

Queries on a totally ordered set S that return a pointer to the element of S with the smallest (for MINIMUM) or largest (for MAXIMUM) key.

SUCCESSOR(S, x)

A query that, given an element x whose key is from a totally ordered set S , returns a pointer to the next larger element in S , or NIL if x is the maximum element.

PREDECESSOR(S, x)

A query that, given an element x whose key is from a totally ordered set S , returns a pointer to the next smaller element in S , or NIL if x is the minimum element.

In some situations, we can extend the queries SUCCESSOR and PREDECESSOR so that they apply to sets with nondistinct keys. For a set on n keys, the normal presumption is that a call to MINIMUM followed by $n - 1$ calls to SUCCESSOR enumerates the elements in the set in sorted order.

We usually measure the time taken to execute a set operation in terms of the size of the set. For example, Chapter 13 describes a data structure that can support any of the operations listed above on a set of size n in $O(\lg n)$ time.

Of course, you can always choose to implement a dynamic set with an array. The advantage of doing so is that the algorithms for the dynamic-set operations are simple. The downside, however, is that many of these operations have a worst-case running time of $\Theta(n)$. If the array is not sorted, INSERT and DELETE can take $\Theta(1)$ time, but the remaining operations take $\Theta(n)$ time. If instead the array is maintained in sorted order, then MINIMUM, MAXIMUM, SUCCESSOR, and PREDECESSOR take $\Theta(1)$ time; SEARCH takes $O(\lg n)$ time if implemented with binary search; but INSERT and DELETE take $\Theta(n)$ time in the worst case. The data structures studied in this part improve on the array implementation for many of the dynamic-set operations.

Overview of Part III

Chapters 10–13 describe several data structures that we can use to implement dynamic sets. We'll use many of these data structures later to construct efficient algorithms for a variety of problems. We already saw another important data structure—the heap—in Chapter 6.

Chapter 10 presents the essentials of working with simple data structures such as arrays, matrices, stacks, queues, linked lists, and rooted trees. If you have taken an introductory programming course, then much of this material should be familiar to you.

Chapter 11 introduces hash tables, a widely used data structure supporting the dictionary operations INSERT, DELETE, and SEARCH. In the worst case, hash tables require $\Theta(n)$ time to perform a SEARCH operation, but the expected time for hash-table operations is $O(1)$. We rely on probability to analyze hash-table operations, but you can understand how the operations work even without probability.

Binary search trees, which are covered in Chapter 12, support all the dynamic-set operations listed above. In the worst case, each operation takes $\Theta(n)$ time on a tree with n elements. Binary search trees serve as the basis for many other data structures.

Chapter 13 introduces red-black trees, which are a variant of binary search trees. Unlike ordinary binary search trees, red-black trees are guaranteed to perform well: operations take $O(\lg n)$ time in the worst case. A red-black tree is a balanced search tree. Chapter 18 in Part V presents another kind of balanced search tree, called a B-tree. Although the mechanics of red-black trees are somewhat intricate, you can glean most of their properties from the chapter without studying the mechanics in detail. Nevertheless, you probably will find walking through the code to be instructive.